

A Deterministic Asynchronous Service-Based Architecture for Agent-Based Transport Simulation

Paul Heinrich^{*1} and Kai Nagel¹

¹Technische Universität Berlin, Chair of Transport Systems Planning and Transport Telematics, Straße des 17. Juni 135, 10623 Berlin

^{*}Correspondence: heinrich@vsp.tu-berlin.de

1 INTRODUCTION

Modern transport systems increasingly rely on dynamic, real-time services such as on-demand mobility and disruption management, posing new requirements for simulation frameworks. Meeting these requirements increasingly depends on the interaction with computationally expensive components such as routing or fleet management. A straightforward approach is to integrate these components directly into the simulation loop using a blocking execution model. This limits scalability, especially if the components block the simulation frequently due to long execution times. Tightly coupled simulation architectures make it difficult to reuse the existing implementations or scale the computational resources independently.

In frameworks like Multi-Agent Transport Simulation (MATSim) [1], this issue arises prominently in within-day replanning, where agents may require computationally expensive routing decisions during simulation execution. To address this challenge, we propose an asynchronous execution model combined with a service-based architecture. Our approach enables simulations to issue requests to external components ahead of time and process them concurrently with simulation execution, while preserving deterministic behavior. By decoupling simulation and external services, computational resources can be scaled independently, and existing implementations can be reused across programming languages.

We implement this approach in the Rust QSim (RQSim), a prototype of MATSim written in Rust, and integrate an external public transit (PT) router from MATSim via gRPC Heinrich et al. [2], Heinrich and Singh [3]. Using a large-scale scenario, we evaluate how asynchronous request issuance and scalable service deployment affect simulation performance. Our simulation implements a single MATSim iteration without scoring.

2 ARCHITECTURE

Since MATSim’s default mobsim doesn’t scale efficiently with the number of threads we have re-implemented a prototype of the mobsim in Rust as RQSim that relies on network domain decomposition [4]. Complexity arises from components plugged into the mobsim that it dynamically interacts with [“within-day replanning”; see also 5]. First experiments in the RQSim with synchronous real-time routing showed strong runtime issues [6].

Use cases in which within-day replanning is employed are for example evacuation studies with MATSim where real-time aspects, such as unpredicted wind direction changes, are important [7]. The simulation of demand-responsive transport does also rely on within-day replanning. To prevent computational bottlenecks, PT simulation and routing in MATSim are simplified by precomputing agent routes prior to the mobsim. Agents who miss a planned departure wait for the next vehicle on the same line serving their destination. Although often suboptimal compared to alternative routes, invoking the PT router during simulation is computationally expensive, but would improve realism.

In order to achieve low simulation runtimes with within-day replanning, we rely on two core principles: calls to components can be executed asynchronously, and external components such as a PT router are deployed as decoupled services. The first requirement allows route calculations to be performed in parallel with the simulation; the second allows flexible resource allocation for the router.

For asynchronous within-day replanning, we implemented a “wake-up mechanism” for agents while they are performing an activity. Based on a flag in the plan, agents are now woken up τ simulation seconds (“lookahead”) before their activity ends. This allows them to interact with the environment, for example by sending routing requests. If a response is not available when the activity ends, simulation progress is paused until it arrives (see Figure 1a). Determinism is preserved because requests are issued and responses are consumed at predefined simulation time steps, independent of their wall-clock completion order, assuming deterministic service responses for identical requests.

To allow horizontal scaling of the external routing service, that is, scaling the number of available router threads, and make the simulation agnostic to the concrete service implementation, we added an adapter thread. This thread receives routing requests from the RQSim, sends them to the external service, receives

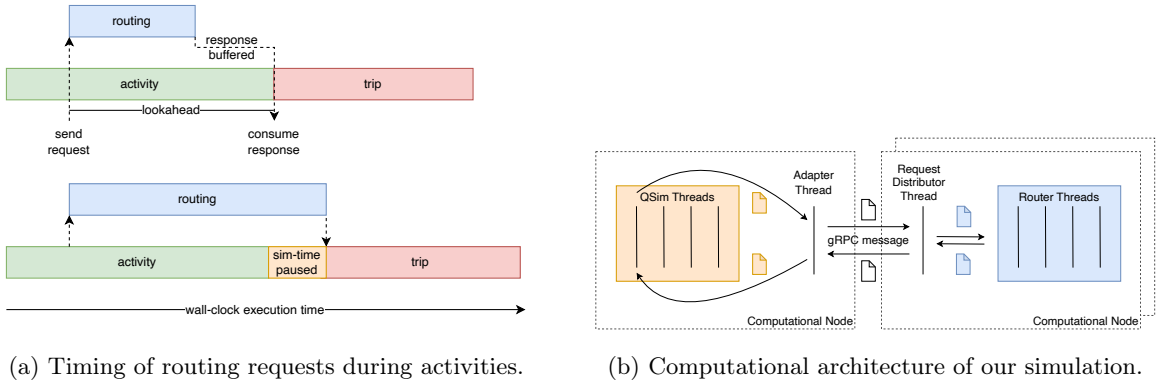


Figure 1: Architecture of our simulation.

Table 1: Real-time ratio (RTR) for different QSim and router configurations.

run id	partitions	sample (%)	lookahead (sim seconds)	routing threads	RTR
1	192	1	0	1	601
2	192	1	0	192	754
3	192	1	200	1	621
4	192	1	200	192	7688
5	192	1	600	1	623
6	192	1	600	192	11684
7	192	1	1800	1	612
8	192	1	1800	192	12171
9	192	10	0	24	243
10	192	10	0	288	231
11	192	10	200	24	973
12	192	10	200	288	4579
13	192	10	600	24	1177
14	192	10	600	288	5016
15	192	10	1800	24	1250
16	192	10	1800	288	4568

the result, and forwards it back to the RQSim. Note that this design can in principle support any kind of service connection to the RQSim, while the RQSim itself does not need to know any implementation details. We decided to use gRPC to connect RQSim with external services because it supports HTTP/2 and offers mature implementations in many programming languages, in particular Java and Rust.

The MATSim library already contains a well-structured and widely used implementation of the RAPTOR algorithm [8]. We encapsulated wrapped the router as a gRPC server. The wrapper server starts a thread pool in which each thread holds an individual instance of the router. Incoming requests are handled by an idle thread. Figure 1b shows the overall system architecture. In our setting, RQSim and routing were executed on different computational nodes. The router was deployed on multiple nodes as needed.

3 EXPERIMENTS

We ran our experiments on the high-performance cluster NHR@ZIB (Nationales Hochleistungsrechnen at Zuse Institute Berlin). Each computational node used an AMD EPYC 9654 CPU @ 3.55GHz (384MB L3 cache, 96 cores, and 192 threads). We use the MATSim Berlin Model v6.4 [9]. We used the calibrated 1% and 10% sample populations (50k/500k agents) and set the minimum activity participation time to 300 s. We varied the lookahead (bounded by the activity duration) and the number of router threads, limiting the latter to 24 per node and adding routing nodes as needed.

As a performance measure, we use the real time ratio (RTR). It indicates how many simulation steps can be executed in one unit of real time. Since our simulation is based on one-second time steps, it expresses how many simulation seconds are performed in one real-time second. In [4], we showed that the prototype levels out at an RTR of 20,000–30,000 without within-day replanning.¹

Table 1 presents the RTRs for different parameter configurations of our simulation runs in the new

¹We consider RTR as more important for transport simulations than, say, speed-up, since we are interested in fast turn-around for smallish scenarios. Speed-up can be arbitrarily improved by making the scenario larger.

setting with asynchronous within-day calls. We can make the following observations:

- Introducing a lookahead substantially boosts the RTR. The jump from a lookahead of 0 to 200 is much larger than the jump from 200 to 600 or from 600 to 1800 (e.g., run 2 vs. 4). In some cases, a larger lookahead even led to a lower RTR. One possible explanation is that, once agents wake up, they are informed by the simulation at every subsequent time step, whereas sleeping agents are not. This requires further investigation.
- Changing the number of router threads has the greatest impact when a lookahead is used (e.g., run 3 vs. 4, 11 vs. 12). If there is no preplanning and thus no asynchronous routing, the simulation is dominated by waiting for the routing requests itself rather than by distribution of requests. We can also see that larger sample sizes require more routing capacity. For the 1% sample, increasing routing capacity from 1 to 192 threads substantially improves performance when lookahead is enabled. The same holds for the 10% sample with an increase to 288 threads.
- The 1% sample reaches higher RTR values than the 10% sample (e.g., run 8 vs. 16). This is probably because the larger sample generates more requests and therefore more requests whose response time exceeds the lookahead, which in turn increases the time during which the simulation slows down.

4 DISCUSSION & LIMITATIONS

Our results show that lookahead and routing capacity are complementary. Additional routing capacity provides little benefit without asynchronous preplanning, while lookahead cannot hide routing latency when service capacity is insufficient. Comparing settings 1 and 8 in Table 1, we observe an improvement by more than a factor of 20. However, in some cases calling the router right at the beginning of an activity is still not enough. Imagine an activity that lasts 600 simulation seconds. With an RTR of 10,000, the simulation needs roughly $\frac{600}{10,000} s = 60$ ms of real time for this interval. If the router takes longer than this, there is no way to prevent the simulation from waiting except by calling the router even earlier, before the activity starts. This would require additional implementation logic. For deterministic modes, where travel time is known in advance, e.g., disturbance-free PT, agents could begin preplanning while still performing a leg. For stochastic modes, such as network modes with congestion like car, agents do not know their activity start and end times in advance and therefore cannot preplan as easily. They could request routes for several likely departure times, but this would increase the number of routing requests.

So far, we have not analyzed how much agents benefit behaviorally from real-time routing. Our prototype currently relies entirely on route calculation on the fly.

5 ACKNOWLEDGEMENTS

The authors gratefully acknowledge the computing time made available to them on the high-performance cluster at the NHR Center NHR@ZIB. This center is jointly supported by the Federal Ministry of Education and Research and the state governments participating in the NHR.

The authors used ChatGPT (OpenAI) for language editing and phrasing suggestions. All technical content, analysis, and conclusions were developed and verified by the authors.

REFERENCES

- [1] A. Horni, K. Nagel, and K. W. Axhausen, editors. *The Multi-Agent Transport Simulation MATSim*. Ubiquity, London, 2016. doi:10.5334/baw.
- [2] Paul Heinrich, Janek Laudan, Andreas Hygrel, Friedrich Völkers, and Dharendra Singh. matsim-vsp/parallel_qsim_rust: v0.3.0, July 2026. URL <https://doi.org/10.5281/zenodo.21722942>.
- [3] Paul Heinrich and Dharendra Singh. paulheirn/parallel-qsim-berlin: v1.0.0, April 2026. URL <https://doi.org/10.5281/zenodo.19731191>.
- [4] Janek Laudan, Paul Heinrich, and Kai Nagel. High-performance mobility simulation: Implementation of a parallel distributed message-passing algorithm for matsim. *Information*, 16(2), 2025. ISSN 2078-2489. doi:10.3390/info16020116. URL <https://www.mdpi.com/2078-2489/16/2/116>.
- [5] C. Dobler and K. Nagel. Within-day replanning. In A. Horni, K. Nagel, and K. W. Axhausen, editors, *The Multi-Agent Transport Simulation MATSim*, chapter 30. Ubiquity, London, 2016. doi:10.5334/baw.
- [6] Paul Heinrich, Janek Laudan, and Kai Nagel. Real-Time Routing in Traffic Simulations: A Distributed Event Processing Approach. In *2024 23rd International Symposium on Parallel and Distributed Computing (ISPDC)*, pages 1–5, 2024. doi:10.1109/ISPDC62236.2024.10705399.
- [7] Dharendra Singh, Lin Padgham, and Kai Nagel. Using MATSim as a component in dynamic agent-based micro-simulations. In Louise A. Dennis, Rafael H. Bordini, and Yves Lespérance, editors, *Engineering Multi-Agent Systems*, pages 85–105, Cham, 2020. Springer International Publishing. ISBN 978-3-030-51417-4.
- [8] M. Rieser, D. Métrailler, and J. Lieberherr. Adding Realism and Efficiency to Public Transportation in MATSim. In *18th Swiss Transport Research Conference*, Monte Verità / Ascona, May 16-18, 2018, May 2018.
- [9] Christian Rakow, Moritz Kreuschner, and Kai Nagel. Advancing the MATSim open Berlin scenario: Improvements in transport scenario generation and calibration methods. *Transportation Research Procedia*, 2024. doi:10.1016/j.trpro.2025.04.091.